

```

# =====
# Install Required Libraries
# =====

!pip install -q transformers datasets evaluate accelerate torch

# =====
# Import Libraries
# =====

import numpy as np
from datasets import Dataset, DatasetDict
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    pipeline
)
import evaluate

print("All required libraries are installed.\n")

# =====
# Create Our Own Dataset
# =====

print("Creating Dataset...\n")

train_data = {
    "text": [
        "I love this movie.",
        "The acting was amazing.",
        "This product is excellent.",
        "The food was delicious.",
        "I really enjoyed this experience.",
        "The laptop performance is fantastic.",
        "I hate this movie.",
        "The service was terrible.",
        "This product is disappointing.",
        "The food tastes awful.",
        "I am not satisfied with the quality.",
        "The laptop is very slow."
    ],
    "label": [
        1,1,1,1,1,1,
        0,0,0,0,0,0
    ]
}

```

```

    ]
}

test_data = {
    "text": [
        "This movie is wonderful.",
        "The film was boring.",
        "I love this product.",
        "The service is terrible.",
        "The food is amazing.",
        "I dislike this laptop."
    ],
    "label": [
        1,0,1,0,1,0
    ]
}

# Create Dataset Objects
dataset = DatasetDict({

    "train": Dataset.from_dict(train_data),

    "test": Dataset.from_dict(test_data)

})

print("Dataset Created Successfully!\n")

# =====
# Load the Tokenizer
# =====

print("Loading DistilBERT Tokenizer...\n")

tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-uncased"
)

# =====
# Tokenization Function
# =====

def tokenize_function(example):

```

```

    return tokenizer(

        example["text"],

        padding="max_length",

        truncation=True,

        max_length=128

    )

# Tokenize Dataset
print("Tokenizing Dataset...\n")

tokenized_dataset = dataset.map(
    tokenize_function,
    batched=True
)

# =====
# Load Pre-trained DistilBERT Model
# =====

print("Loading DistilBERT Model...\n")

model = AutoModelForSequenceClassification.from_pretrained(

    "distilbert-base-uncased",

    num_labels=2

)

# =====
# Load Accuracy Metric
# =====

accuracy = evaluate.load("accuracy")

def compute_metrics(eval_pred):

    logits, labels = eval_pred

    predictions = np.argmax(

```

```

        logits,
        axis=1
    )

    return accuracy.compute(
        predictions=predictions,
        references=labels
    )

# =====
# Training Arguments
# =====

training_args = TrainingArguments(

    output_dir="./results",

    learning_rate=2e-5,

    per_device_train_batch_size=4,

    per_device_eval_batch_size=4,

    num_train_epochs=3,

    weight_decay=0.01,

    logging_steps=10,

    report_to="none"
)

# =====
# Create Trainer Object
# =====

trainer = Trainer(

    model=model,

    args=training_args,

    train_dataset=tokenized_dataset["train"],

    eval_dataset=tokenized_dataset["test"],

```

```

        compute_metrics=compute_metrics
    )

# =====
# Train the Model
# =====

print("Training Started...\n")

trainer.train()

print("\nTraining Completed Successfully!")

# =====
# Evaluate the Model
# =====

print("\nEvaluating the Model...\n")

results = trainer.evaluate()

print("\nEvaluation Results:")
print(results)

# =====
# Create Prediction Pipeline
# =====

classifier = pipeline(

    "sentiment-analysis",

    model=trainer.model,

    tokenizer=tokenizer
)

# =====
# Single Sentence Prediction
# =====

print("\n=====")
print("SINGLE SENTENCE PREDICTION")

```

```

print("=====\n")

print(
    classifier(
        "The movie was absolutely wonderful!"
    )
)

print(
    classifier(
        "The film was boring and disappointing."
    )
)

# =====
# Multiple Sentence Prediction
# =====

print("\n=====")
print("MULTIPLE SENTENCE PREDICTION")
print("=====\n")

sentences = [

    "The movie was absolutely wonderful!",

    "The film was boring and disappointing.",

    "I love this product.",

    "The service was terrible.",

    "The food was delicious.",

    "The laptop performance is excellent.",

    "I am not satisfied with the quality.",

    "This is the best experience I have ever had."

]

```

```
predictions = classifier(sentences)

for sentence, prediction in zip(
    sentences,
    predictions):

    print("Sentence :", sentence)
    print("Prediction :", prediction)
    print("-" * 60)
```

All required libraries are installed.

Creating Dataset...

Dataset Created Successfully!

Loading DistilBERT Tokenizer...

config.json: 100%

483/483 [00:00<00:00, 21.0kB/s]

tokenizer_config.json: 100%

48.0/48.0 [00:00<00:00, 2.22kB/s]

vocab.txt: 100%

232k/232k [00:00<00:00, 2.55MB/s]

tokenizer.json: 100%

466k/466k [00:00<00:00, 8.34MB/s]

Tokenizing Dataset...

Map: 100%

12/12 [00:00<00:00, 179.69 examples/s]

Map: 100%

6/6 [00:00<00:00, 104.43 examples/s]

Loading DistilBERT Model...

model.safetensors: reconstructing file: 100%

268MB / 268MB, 21.0MB/s

model.safetensors: downloading bytes:

250MB, 23.1MB/s

Loading weights: 100%

```

100/100 [00:00<00:00, 2616.19it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased
Key | Status |
-----+-----+
vocab_projector.bias | UNEXPECTED |
vocab_layer_norm.bias | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
classifier.weight | MISSING |
pre_classifier.bias | MISSING |
classifier.bias | MISSING |
pre_classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from
the checkpoint. Consider training on your downstream task.

Downloading builder script:

4.20k/? [00:00<00:00, 264kB/s]
Training Started...

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but no accelerator is
found, then device pinned memory won't be used.
  super().__init__(loader)
[9/9 00:35, Epoch 3/3]

```

Step Training Loss

```

Writing model shards: 100%

1/1 [00:00<00:00, 1.44it/s]

Training Completed Successfully!

Evaluating the Model...

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but no accelerator is
found, then device pinned memory won't be used.
  super().__init__(loader)
[2/2 00:00]

```

Training Loss Validation Loss Step Accuracy

No log	0.659465	9	0.500000
--------	----------	---	----------

```

Evaluation Results:
{'eval_loss': 0.6594645977020264, 'eval_accuracy': 0.5}

```

```

=====
SINGLE SENTENCE PREDICTION
=====

```



```
[{'label': 'LABEL_0', 'score': 0.507583737373352}]  
[{'label': 'LABEL_0', 'score': 0.522341787815094}]
```

```
=====
```

MULTIPLE SENTENCE PREDICTION

```
=====
```

```
Sentence : The movie was absolutely wonderful!  
Prediction : {'label': 'LABEL_0', 'score': 0.507583737373352}  
-----  
Sentence : The film was boring and disappointing.  
Prediction : {'label': 'LABEL_0', 'score': 0.522341787815094}  
-----  
Sentence : I love this product.  
Prediction : {'label': 'LABEL_0', 'score': 0.5090335607528687}  
-----  
Sentence : The service was terrible.  
Prediction : {'label': 'LABEL_0', 'score': 0.5629377365112305}  
-----  
Sentence : The food was delicious.  
Prediction : {'label': 'LABEL_0', 'score': 0.5016550421714783}  
-----  
Sentence : The laptop performance is excellent.  
Prediction : {'label': 'LABEL_0', 'score': 0.5067023038864136}  
-----  
Sentence : I am not satisfied with the quality.  
Prediction : {'label': 'LABEL_0', 'score': 0.5303062200546265}  
-----  
Sentence : This is the best experience I have ever had.  
Prediction : {'label': 'LABEL_0', 'score': 0.5115489959716797}  
-----
```